

REQUERIMIENTO TÉCNICO: Módulo de Campaña Solidaria

Documentación de Producción / Arquitectura Frontend Modular

Propósito: Implementar un flujo UI desacoplado para visibilizar el apoyo humanitario tras el reciente sismo en Venezuela, permitiendo el despliegue dinámico de una interfaz (Modal o Banner) sin introducir regresiones en el ciclo de vida del *opening/loader* ni incrustar URLs fijas en el marcado de vistas.

1. Configuración Centralizada (Diccionario de URLs)

Se requiere que las URLs de destino y las banderas de estado estén completamente aisladas del componente de presentación. Esto garantiza la escalabilidad si se requiere actualizar las plataformas de recolección en caliente:

```
// config/solidarityCampaign.js (o variables de entorno correspondientes)
export const SOLIDARITY_CONFIG = {
  isActive: true, // Switch maestro para activar/desactivar la campaña
  primaryDonationUrl: "https://caritasvenezuela.org", // Canal principal (Cáritas)
  secondaryDonationUrl: "https://cruzrojavenezolana.org", // Canal secundario (Cruz Roja)
  analyticsEventName: "click_donation_venezuela" // Evento de tracking para analítica
};
```

2. Copys Homologados (Presentación de Contenido)

Opción A: Estructura del Modal Post-Opening

Unidos por Venezuela

Nuestro país nos necesita. Ante el reciente terremoto que ha afectado a tantas familias, hemos decidido aportar nuestro grano de arena.

El 100% de los ingresos recaudados a través de esta página web será donado directamente a las familias que lo necesiten.

Si deseas colaborar de manera directa o conocer los canales oficiales de apoyo y centros de acopio, puedes hacerlo a través del siguiente enlace:

[Botón Principal] -> Consume SOLIDARITY_CONFIG.primaryDonationUrl

[Acción de Cierre] -> Continuar al sitio

Opción B: Estructura del Banner Superior Fijo

 **Campaña de Apoyo:** El 100% de lo recaudado en este sitio será destinado a las familias afectadas por el terremoto en Venezuela. [\[Conoce cómo ayudar o donar aquí\]](#) -> Enlaza a `SOLIDARITY_CONFIG.primaryDonationUrl`

3. Directrices de Ingeniería y Auditoría de Archivos

Antes de proceder con el montaje de los componentes, realice el análisis de impacto en los siguientes módulos del core:

- **Ciclo de Vida del Opening:** Inspeccione el archivo encargado del loader inicial (ej. `loader.js`, `useOpening.ts`). El renderizado de la **Opción A** debe encadenarse exclusivamente al resolver la Promesa o invocar el callback `onComplete` de la animación. Queda estrictamente prohibido el uso de temporizadores rígidos (`setTimeout`) desalineados del estado real del DOM.
- **Layout y Geometría de Pantalla (Opción B):** Si se opta por el Banner, asigne la altura del componente a una propiedad personalizada en CSS (ej. `--banner-height: 40px;`) aplicada al contenedor raíz. Esto evitará solapamientos con cabeceras fijas de navegación y mantendrá intactos los cálculos basados en `100vh`.
- **Persistencia de Estado:** Implemente una llave de control en `localStorage` (ej. `ve_earthquake_dismissed`) al ejecutar la acción de cierre. Esto evitará interrupciones intrusivas durante la navegación interna del usuario en la misma sesión.

Requisito de Seguridad Obligatorio (Tabnabbing): Toda etiqueta de hipervínculo que apunte a las URLs externas (Cáritas o Cruz Roja) debe incluir explícitamente los atributos `target="_blank"` y `rel="noopener noreferrer"` para mitigar vulnerabilidades de secuestro de contexto en hilos de ejecución secundarios.

4. Consumo Semántico de Enlaces

El botón de acción debe mapear dinámicamente las propiedades de la configuración inyectada:

```
// Implementación limpia del control de salida externa
typeof trackEvent === 'function' && trackEvent(SOLIDARITY_CONFIG.analyticsEventName){
  className="btn-donation-primary"
}
>
  Ver Canales de Ayuda / Donar Directo
```